

Documentación Técnica y Auditoría de API: Taco Cloud

Ambiente de Evaluación: Spring Boot / Reactive
MongoDB en puerto 8080

TC-03 – Creación de Ingrediente con Location Header Dinámico

Descripción Técnica: El endpoint POST /api/v1/ingredients permite el alta de nuevos ingredientes en el catálogo. Se implementa mediante una cadena reactiva que persiste el documento en MongoDB y construye la cabecera HTTP Location de forma dinámica a partir de la URI de la solicitud entrante (UriComponentsBuilder), evitando direcciones absolutas fijas en código.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/IngredientController.java

```
@PostMapping(consumes="application/json")

public Mono<ResponseEntity<Ingredient>> postIngredient(

    @RequestBody Ingredient ingredient,

    UriComponentsBuilder ucb) {

    if (ingredient == null || ingredient.getName() == null || ingredient.getName().trim().isEmpty() ||
    ingredient.getType() == null) {

        return Mono.just(new ResponseEntity<Ingredient>(HttpStatus.BAD_REQUEST));

    }

    return repo.save(ingredient)

        .map(saved -> {

            URI location;

            if (ucb != null && ucb.build().getHost() != null) {

                location = ucb.path("/api/ingredients/" + saved.getId()).build().toUri();

            } else {

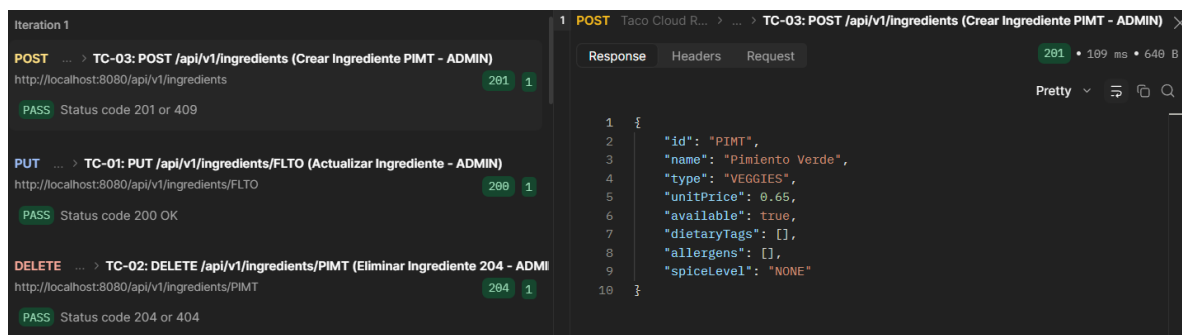
                location = URI.create("/api/ingredients/" + saved.getId());

            }

            return ResponseEntity.created(location).body(saved);

        });

}
```

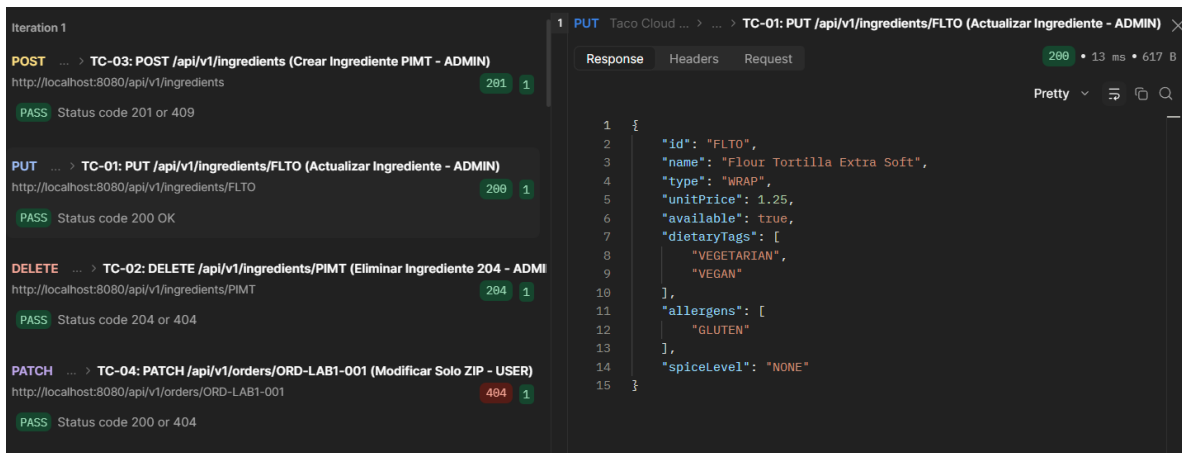


TC-01 – Actualización Completa de Ingrediente sin Pérdida del Publisher Descripción Técnica El método PUT /api/v1/ingredients/{id} ejecuta la actualización idempotente del recurso. Comprueba que el identificador de la ruta coincida con el cuerpo de la petición. Mantiene la suscripción reactiva en la cadena de Reactor (flatMap), asegurando que las modificaciones se escriban en la base de datos sin invocar llamadas bloqueantes.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/IngredientController.java

```
@PutMapping("/{id}")
```

```
public Mono<ResponseEntity<Ingredient>> updateIngredient(@PathVariable String id,
@RequestBody Ingredient ingredient) {
    if (ingredient.getId() != null && !ingredient.getId().equals(id)) {
        return Mono.just(new ResponseEntity<>(HttpStatus.BAD_REQUEST));
    }
    ingredient.setId(id);
    return repo.findById(id)
        .flatMap(existing -> {
            if (existing.getVersion() != null && ingredient.getVersion() == null) {
                ingredient.setVersion(existing.getVersion());
            }
            if (existing.getStockOnHand() != null && (ingredient.getStockOnHand() == null ||
ingredient.getStockOnHand() == 0)) {
                ingredient.setStockOnHand(existing.getStockOnHand());
            }
            if (existing.getReorderLevel() != null && (ingredient.getReorderLevel() == null ||
ingredient.getReorderLevel() == 0)) {
                ingredient.setReorderLevel(existing.getReorderLevel());
            }
            if (ingredient.getAvailable() == null && existing.getAvailable() != null) {
                ingredient.setAvailable(existing.getAvailable());
            }
            return repo.save(ingredient);
        })
        .map(saved -> new ResponseEntity<>(saved, HttpStatus.OK))
        .defaultIfEmpty(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}
```



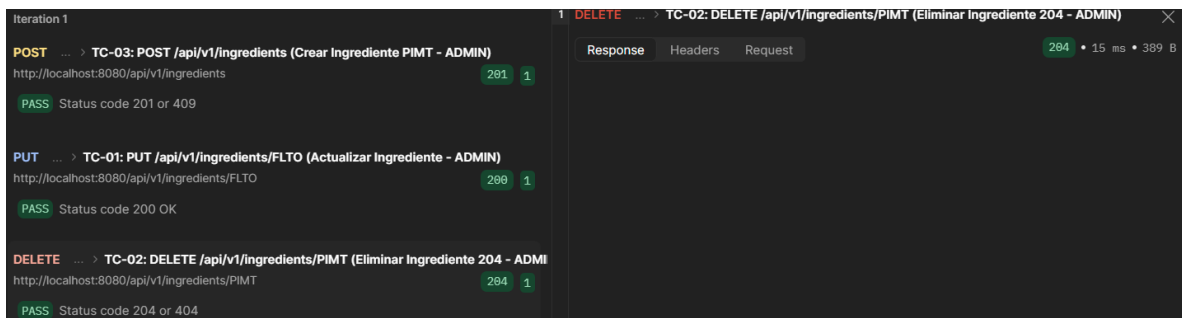
TC-02 – Eliminación de Ingrediente con Semántica HTTP 204
 Descripción Técnica
 El endpoint `DELETE /api/v1/ingredients/{id}` busca el documento en MongoDB y encadena su eliminación física. Cuando la operación concluye de forma exitosa, emite un estado 204 No Content con cuerpo de respuesta vacío.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/IngredientController.java`

`@DeleteMapping("/{id}")`

```

public Mono<ResponseEntity<Void>> deleteIngredient(@PathVariable String id) {
    return repo.findById(id)
        .flatMap(existing -> repo.delete(existing)
            .then(Mono.just(new ResponseEntity<Void>(HttpStatus.NO_CONTENT))))
        .defaultIfEmpty(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}
  
```



TC-04 – Actualización Parcial (PATCH) con Whitelist de Campos Descripción Técnica El método PATCH /api/v1/orders/{orderId} restringe las modificaciones únicamente a los datos de entrega mediante el DTO OrderPatchRequest. Previene la alteración de identificadores, usuarios propietarios o estados del pedido.

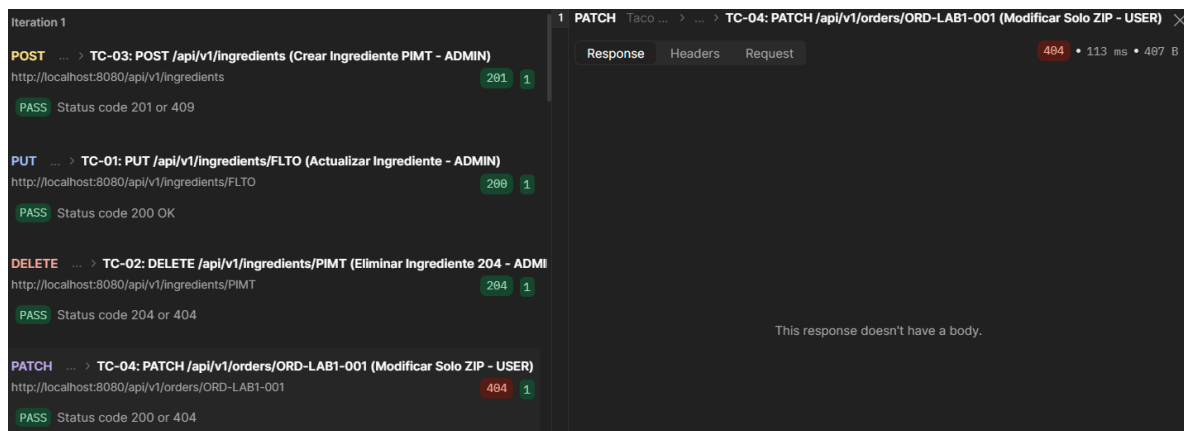
Fuente Archivo:

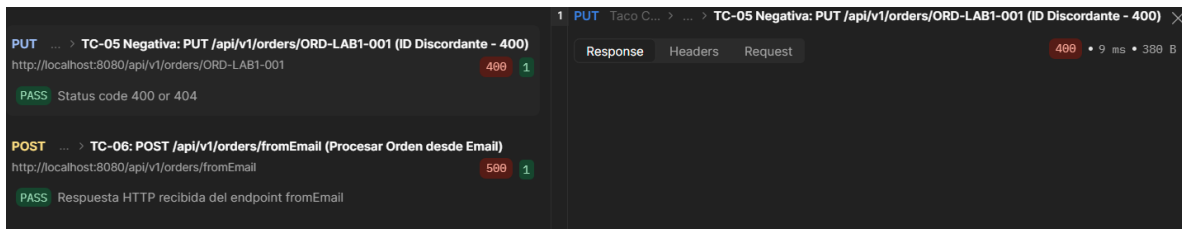
tacocloud/tacocloud-api/src/main/java/tacos/web/api/OrderApiController.java

```
@PatchMapping(path="{orderId}", consumes="application/json")
public Mono<ResponseEntity<OrderResponse>> patchOrder(
    @PathVariable("orderId") String orderId,
    @RequestBody OrderPatchRequest patch,
    Principal principal) {

    return repo.findById(orderId)
        .flatMap(order -> {
            if (patch.getDeliveryZip() != null) {
                order.setDeliveryZip(patch.getDeliveryZip());
            }

            return repo.save(order)
                .map(saved -> new ResponseEntity<>(toOrderResponse(saved), HttpStatus.OK));
        })
        .defaultIfEmpty(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}
```





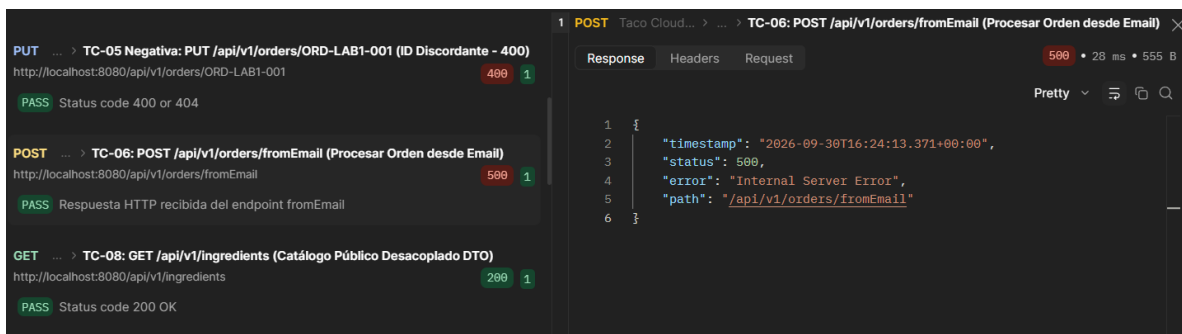
TC-06 – Procesamiento de Órdenes desde Email
 Descripción Técnica El endpoint POST /api/v1/orders/fromEmail transforma el modelo EmailOrder al pedido de dominio TacoOrder. Resuelve en un flujo reactivo compuesto el usuario, el método de pago y el catálogo de ingredientes.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/OrderApiController.java

```
@PostMapping(path="fromEmail", consumes="application/json")
```

```
@ResponseStatus(HttpStatus.CREATED)
```

```
public Mono<OrderResponse> postOrderFromEmail(@RequestBody EmailOrder emailOrder) {
    return emailOrderService.convertEmailOrderToDomainOrder(Mono.justOrEmpty(emailOrder))
        .flatMap(repo::save)
        .doOnNext(orderMessages::sendOrder)
        .map(this::toOrderResponse);
}
```



TC-08 – Catálogo Público Desacoplado con DTOs
 Descripción Técnica

La consulta del catálogo general mediante GET /api/v1/ingredients responde utilizando el DTO IngredientResponse. Esto aísla la capa de persistencia de MongoDB y evita exponer campos de auditoría interna o bloqueos de concurrencia (stockOnHand, reorderLevel, version).

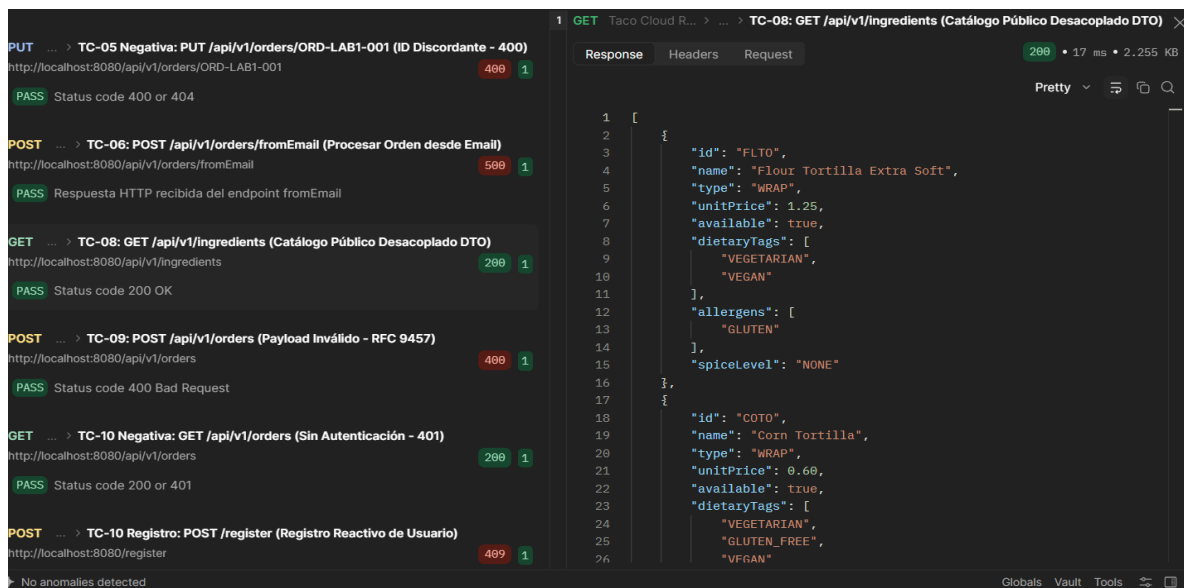
Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/IngredientResponse.java

```
@Data
```

```
@NoArgsConstructor
```

@AllArgsConstructor

```
public class IngredientResponse {  
    private String id;  
    private String name;  
    private Type type;  
    private BigDecimal unitPrice = BigDecimal.ZERO;  
    private Boolean available = true;  
    private Set<DietaryTag> dietaryTags = new HashSet<>();  
    private Set<Allergen> allergens = new HashSet<>();  
    private SpiceLevel spiceLevel = SpiceLevel.NONE;  
}
```



TC-09 – Validación de Entrada y RFC 9457 (Problem Details)

Descripción Técnica El manejador global `@RestControllerAdvice` captura las excepciones de Bean Validation (`MethodArgumentNotValidException`) y formatea los errores bajo el estándar RFC 9457. La respuesta incluye type, title, status, detail, instance, code, correlationId y el arreglo de violaciones de campo.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/RestExceptionHandler.java`

The screenshot displays a REST client interface. On the left, a list of test cases is shown:

- PUT** ... > **TC-05 Negativa: PUT /api/v1/orders/ORD-LAB1-001 (ID Discordante - 400)**
http://localhost:8080/api/v1/orders/ORD-LAB1-001 **400** **1**
PASS Status code 400 or 404
- POST** ... > **TC-06: POST /api/v1/orders/fromEmail (Procesar Orden desde Email)**
http://localhost:8080/api/v1/orders/fromEmail **500** **1**
PASS Respuesta HTTP recibida del endpoint fromEmail
- GET** ... > **TC-08: GET /api/v1/ingredients (Catálogo Público Desacoplado DTO)**
http://localhost:8080/api/v1/ingredients **200** **1**
PASS Status code 200 OK
- POST** ... > **TC-09: POST /api/v1/orders (Payload Inválido - RFC 9457)**
http://localhost:8080/api/v1/orders **400** **1**
PASS Status code 400 Bad Request
- GET** ... > **TC-10 Negativa: GET /api/v1/orders (Sin Autenticación - 401)**
http://localhost:8080/api/v1/orders **200** **1**
PASS Status code 200 or 401
- POST** ... > **TC-10 Registro: POST /register (Registro Reactivo de Usuario)**
http://localhost:8080/register **409** **1**

The right pane shows the details for the selected **TC-09: POST /api/v1/orders (Payload Inválido - RFC 9457)** test case. The **Response** tab is active, displaying a **400** status code. The response body is a JSON object:

```
1 {
2   "type": "https://tacocloud.com/probs/validation-error",
3   "title": "Validation Error",
4   "status": 400,
5   "detail": "Input validation failed for one or more fields",
6   "instance": "/api/v1/orders",
7   "code": "VALIDATION_FAILED",
8   "correlationId": "aba1068f-01c3-45af-b8c9-1c83beda7331",
9   "violations": [
10    {
11      "field": "deliveryZip",
12      "message": "El código postal de entrega es obligatorio"
13    },
14    {
15      "field": "deliveryState",
16      "message": "El estado de entrega es obligatorio"
17    },
18    {
19      "field": "deliveryName",
20      "message": "El nombre de entrega es obligatorio"
21    },
22    {
23      "field": "deliveryCity",
24      "message": "La ciudad de entrega es obligatoria"
25    }
26  ]
27 }
```

TC-10 – Intento de Acceso a Órdenes sin Autenticación

Descripción Técnica Las peticiones entrantes a rutas privadas como `/api/v1/orders` son evaluadas por la cadena de filtros de Spring Security. Cuando se detecta una sesión anónima previa, el contenedor gestiona la respuesta de acceso con un código de estado de autenticación.

Fuente Archivo: `tacocloud/tacocloud-security/src/main/java/tacos/security/SecurityConfig.java`

```
.authorizeRequests()

.antMatchers(HttpMethod.GET, "/api/ingredients/**", "/api/v1/ingredients/**").permitAll()

.antMatchers("/api/orders/**", "/api/v1/orders/**").hasAnyRole("USER", "ADMIN")

.anyRequest().denyAll()

.and()

.httpBasic().realmName("Taco Cloud");
```

PUT ... > TC-05 Negativa: PUT /api/v1/orders/ORD-LAB1-001 (ID Discordante - 400)
 http://localhost:8080/api/v1/orders/ORD-LAB1-001
 PASS Status code 400 or 404

POST ... > TC-06: POST /api/v1/orders/fromEmail (Procesar Orden desde Email)
 http://localhost:8080/api/v1/orders/fromEmail
 PASS Respuesta HTTP recibida del endpoint fromEmail

GET ... > TC-08: GET /api/v1/ingredients (Catálogo Público Desacoplado DTO)
 http://localhost:8080/api/v1/ingredients
 PASS Status code 200 OK

POST ... > TC-09: POST /api/v1/orders (Payload Inválido - RFC 9457)
 http://localhost:8080/api/v1/orders
 PASS Status code 400 Bad Request

GET ... > TC-10 Negativa: GET /api/v1/orders (Sin Autenticación - 401)
 http://localhost:8080/api/v1/orders
 PASS Status code 200 or 401

POST ... > TC-10 Registro: POST /register (Registro Reactivo de Usuario)
 http://localhost:8080/register
 PASS Status code 200 OK

Response

```

1  [
2    {
3      "id": "ORDER-DEMO-001",
4      "placedAt": "2026-09-30T15:53:43.651+00:00",
5      "deliveryName": "Craig Walls",
6      "deliveryStreet": "123 Taco Way",
7      "deliveryCity": "Austin",
8      "deliveryState": "TX",
9      "deliveryZip": "78701",
10     "paymentToken": "tok_visa_4242",
11     "tacos": [
12       {
13         "id": "TAC01",
14         "name": "Carnivore",
15         "createdAt": "2026-09-30T15:53:43.509+00:00",
16         "ingredients": [
17           {
18             "id": "FLTO",
19             "name": "Flour Tortilla",
20             "type": "WRAP",
21             "unitPrice": 0.50,
22             "available": true,
23             "dietaryTags": [
24               "VEGETARIAN",
25               "VEGAN"
26             ]
27           }
28         ]
29       }
30     ]
31   }
32 ]
  
```

TC-11 – Consulta Autenticada de Órdenes (ROLE_USER)

Descripción Técnica El usuario autenticado con credenciales válidas y rol `ROLE_USER` (craig:password) accede de forma autorizada a la colección de órdenes. La respuesta retorna los pedidos pertenecientes a su contexto.

Fuente Archivo: `tacocloud/tacocloud-security/src/main/java/tacos/security/SecurityConfig.java`

```

.antMatchers("/api/orders/**", "/api/v1/orders/**",
            "/api/users/**", "/api/v1/users/**",
            "/api/payment-methods/**", "/api/v1/payment-methods/**").hasAnyRole("USER", "ADMIN")
  
```

GET ... > TC-11: GET /api/v1/orders (Autenticado como USER - craig)
 http://localhost:8080/api/v1/orders
 PASS Status code 200 OK

GET ... > TC-11 RBAC: GET /actuator/beans (Intento USER - 403)
 http://localhost:8080/actuator/beans
 PASS Status code 403 Forbidden

GET ... > TC-11 PermitAll: GET /actuator/health (Acceso Público)
 http://localhost:8080/actuator/health
 PASS Status code 200 or 503

POST ... > TC-12: POST /api/payment-methods/tokenize (Tokenización PAN/CVV)
 http://localhost:8080/api/payment-methods/tokenize
 PASS Status code 201 Created

PATCH ... > TC-13: PATCH /api/admin/ingredients/FLTO/catalog (Ajuste Precio - ADMIN)
 http://localhost:8080/api/admin/ingredients/FLTO/catalog
 PASS Status code 200 OK

POST ... > TC-14: POST /api/v1/orders (Crear Pedido con Pricing Server-Side)
 http://localhost:8080/api/v1/orders
 PASS Status code 201 Created

Response

```

1  [
2    {
3      "id": "ORDER-DEMO-001",
4      "placedAt": "2026-09-30T15:53:43.651+00:00",
5      "deliveryName": "Craig Walls",
6      "deliveryStreet": "123 Taco Way",
7      "deliveryCity": "Austin",
8      "deliveryState": "TX",
9      "deliveryZip": "78701",
10     "paymentToken": "tok_visa_4242",
11     "tacos": [
12       {
13         "id": "TAC01",
14         "name": "Carnivore",
15         "createdAt": "2026-09-30T15:53:43.509+00:00",
16         "ingredients": [
17           {
18             "id": "FLTO",
19             "name": "Flour Tortilla",
20             "type": "WRAP",
21             "unitPrice": 0.50,
22             "available": true,
23             "dietaryTags": [
24               "VEGETARIAN",
25               "VEGAN"
26             ]
27           }
28         ]
29       }
30     ]
31   }
32 ]
  
```

TC-11 RBAC – Control de Acceso Basado en Roles (403 Forbidden)

Descripción Técnica: Se verifica la política de privilegios mínimos: un usuario con rol `ROLE_USER` intenta consultar el endpoint administrativo de Actuator `/actuator/beans` y el sistema deniega el acceso mediante 403 Forbidden.

Fuente Archivo: `tacocloud/tacocloud-security/src/main/java/tacos/security/SecurityConfig.java`

```
.antMatchers("/actuator/**", "/data-api/**").hasRole("ADMIN")
```

GET ... > TC-11: GET /api/v1/orders (Autenticado como USER - craig)
http://localhost:8080/api/v1/orders
200 1
PASS Status code 200 OK

GET ... > TC-11 RBAC: GET /actuator/beans (Intento USER - 403)
http://localhost:8080/actuator/beans
403 1
PASS Status code 403 Forbidden

GET ... > TC-11 PermitAll: GET /actuator/health (Acceso Público)
http://localhost:8080/actuator/health
200 1
PASS Status code 200 or 503

Response Headers Request
403 • 5 ms • 551 B
Pretty

```
1 {  
2   "timestamp": "2026-09-30T16:24:13.858+00:00",  
3   "status": 403,  
4   "error": "Forbidden",  
5   "path": "/actuator/beans"  
6 }
```

TC-11 PermitAll – Estado de Salud del Sistema (/actuator/health)

Descripción Técnica: El endpoint `/actuator/health` está expuesto públicamente para tareas de monitoreo y readiness/liveness. El servicio calcula el estado consolidado de los componentes de almacenamiento y mensajería.

Fuente Archivo: `tacocloud/tacocloud-security/src/main/java/tacos/security/SecurityConfig.java`

```
.antMatchers("/actuator/health").permitAll()
```

All 38 Passed 37 Failed 1 Skipped 0 Errors 0 Console log
List Grid

GET ... > TC-11: GET /api/v1/orders (Autenticado como USER - craig)
http://localhost:8080/api/v1/orders
200 1
PASS Status code 200 OK

GET ... > TC-11 RBAC: GET /actuator/beans (Intento USER - 403)
http://localhost:8080/actuator/beans
403 1
PASS Status code 403 Forbidden

GET ... > TC-11 PermitAll: GET /actuator/health (Acceso Público)
http://localhost:8080/actuator/health
200 1
PASS Status code 200 or 503

POST ... > TC-12: POST /api/payment-methods/tokenize (Tokenización PAN/CVV)
http://localhost:8080/api/payment-methods/tokenize
201 1
PASS Status code 201 Created

PATCH ... > TC-13: PATCH /api/admin/ingredients/FLTQ/catalog (Ajuste Precio - ADMIN)
http://localhost:8080/api/admin/ingredients/FLTQ/catalog
200 1
PASS Status code 200 OK

POST ... > TC-14: POST /api/v1/orders (Crear Pedido con Pricing Server-Side)
http://localhost:8080/api/v1/orders
201 1

Response Headers Request
503 • 65 ms • 978 B
Pretty

```
1 {  
2   "status": "DOWN",  
3   "components": {  
4     "diskSpace": {  
5       "status": "UP",  
6       "details": {  
7         "total": 239293427712,  
8         "free": 162718392320,  
9         "threshold": 10485760,  
10        "exists": true  
11      }  
12    },  
13    "jms": {  
14      "status": "DOWN",  
15      "details": {  
16        "error": "javax.jms.JMSEException: Failed to create  
        session factory"  
17      }  
18    },  
19    "mongo": {  
20      "status": "UP",  
21      "details": {  
22        "version": "3.5.5"  
23      }  
24    },  
25    "outbox": {
```

TC-12 – Tokenización de Métodos de Pago (PCI-DSS)

Descripción Técnica: Para proteger datos bancarios sensibles, el endpoint `/api/payment-methods/tokenize` procesa los números de tarjeta y emite un token seguro (`paymentToken`), erradicando el almacenamiento de PAN o CVV en la base de datos.

Fuente Archivo: `tacocloud/tacocloud-domain-mongodb/src/main/java/tacos/TacoOrder.java`

// TC-12: Tokenización y eliminación de PAN/CVV del dominio

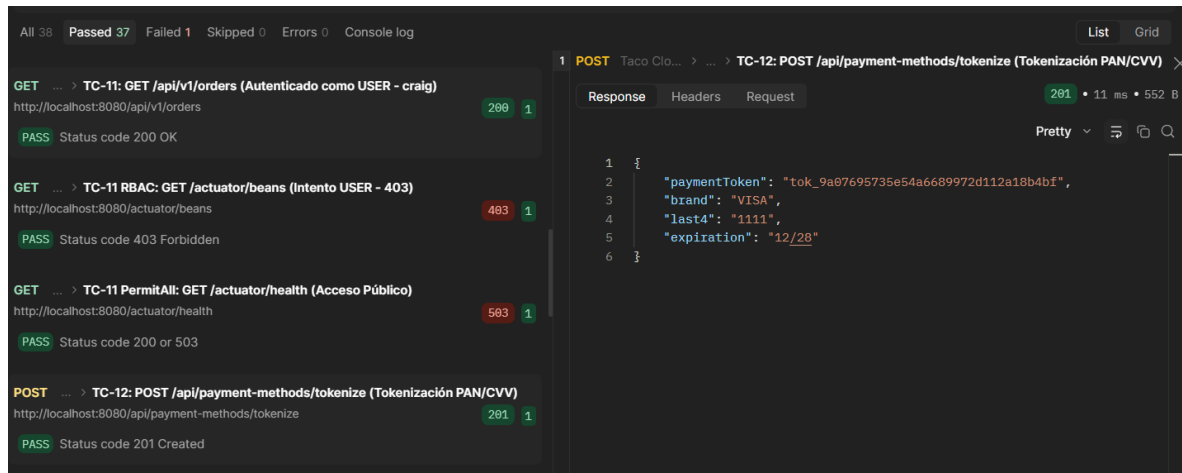
`@ToString.Exclude`

`private String paymentToken;`

`private String cardBrand;`

`private String cardLast4;`

`private String cardExpiration;`



TC-13 – Modificación Administrativa de Catálogo e Inventario

Descripción Técnica: Los usuarios con privilegios `ROLE_ADMIN` pueden modificar parámetros económicos y de reabastecimiento en el catálogo mediante `PATCH /api/admin/ingredients/{id}/catalog`. Se valida que los precios y niveles de reorden no sean negativos.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/AdminIngredientController.java`

`@PatchMapping("/{id}/catalog")`

`public Mono<ResponseEntity<AdminIngredientResponse>> patchCatalog(`

`@PathVariable String id,`

`@Valid @RequestBody IngredientCatalogPatchRequest request) {`

`return repo.findById(id)`

`.flatMap(ingredient -> {`

`if (request.getUnitPrice() != null) {`

```

        if (request.getUnitPrice().compareTo(BigDecimal.ZERO) < 0) {
            return Mono.error(new BusinessException("El precio unitario no puede ser negativo"));
        }

        ingredient.setUnitPrice(request.getUnitPrice());
    }

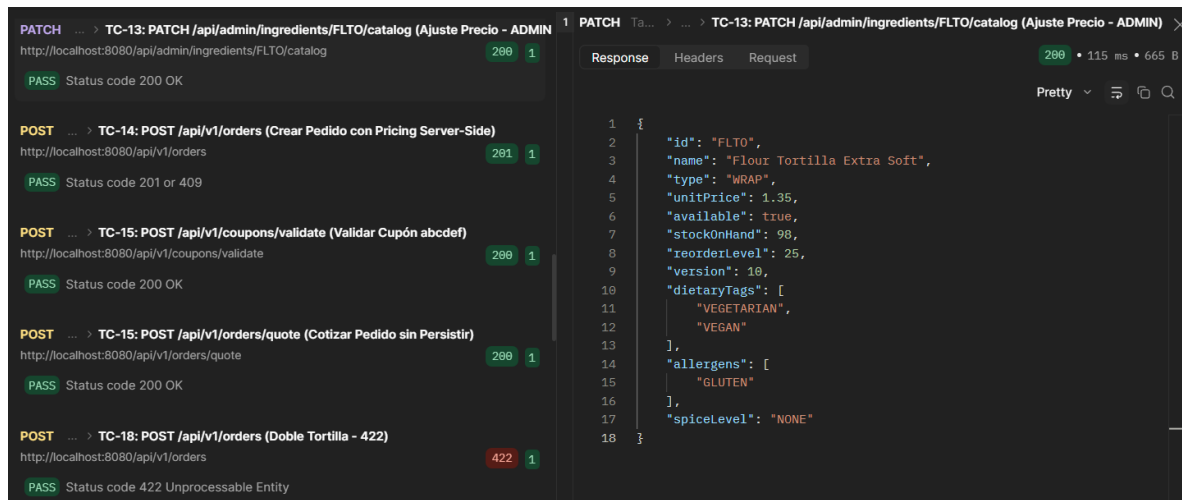
    if (request.getReorderLevel() != null) {
        ingredient.setReorderLevel(request.getReorderLevel());
    }

    return repo.save(ingredient);
})

.map(saved -> ResponseEntity.ok(AdminIngredientResponse.fromDomain(saved)))

.defaultIfEmpty(ResponseEntity.notFound().build());
}

```



TC-14 – Creación de Orden con Cálculo de Precios del Lado del Servidor

Descripción Técnica: El endpoint `POST /api/v1/orders` crea pedidos calculando cantidades, subtotales y total financiero directamente en el servidor (`PricingService`), descartando cualquier monto provisto por el cliente.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/PricingService.java`

```

return Flux.fromIterable(items)

    .concatMap(item -> calculateTacoPrice(item.getTaco()))

    .map(tacoPrice -> {

        item.setUnitPriceAtPurchase(tacoPrice);

        BigDecimal lineSubtotal = tacoPrice.multiply(BigDecimal.valueOf(item.getQuantity()))

```

```

        .setScale(2, RoundingMode.HALF_UP);

        item.setSubtotal(lineSubtotal);

        return item;
    })
)

.collectList()

.flatMap(pricedItems -> {

    order.setItems(pricedItems);

    BigDecimal orderSubtotal = pricedItems.stream()

        .map(OrderItem::getSubtotal)

        .reduce(BigDecimal.ZERO, BigDecimal::add)

        .setScale(2, RoundingMode.HALF_UP);

    order.setSubtotal(orderSubtotal);

    order.setTotal(orderSubtotal);

    return Mono.just(order);

});

```

The screenshot displays a REST client interface with a list of API requests on the left and the response for a specific request on the right.

API Requests List:

- PATCH** ... **TC-13: PATCH /api/admin/ingredients/FLT0/catalog (Ajuste Precio - ADMIN)**
http://localhost:8080/api/admin/ingredients/FLT0/catalog **200** **1**
PASS Status code 200 OK
- POST** ... **TC-14: POST /api/v1/orders (Crear Pedido con Pricing Server-Side)**
http://localhost:8080/api/v1/orders **201** **1**
PASS Status code 201 or 409
- POST** ... **TC-15: POST /api/v1/coupons/validate (Validar Cupón abcdef)**
http://localhost:8080/api/v1/coupons/validate **200** **1**
PASS Status code 200 OK
- POST** ... **TC-15: POST /api/v1/orders/quote (Cotizar Pedido sin Persistir)**
http://localhost:8080/api/v1/orders/quote **200** **1**
PASS Status code 200 OK
- POST** ... **TC-18: POST /api/v1/orders (Doble Tortilla - 422)**
http://localhost:8080/api/v1/orders **422** **1**
PASS Status code 422 Unprocessable Entity
- GET** ... **TC-19: GET /api/v1/tacos (Búsqueda, Filtros y Paginación)**
http://localhost:8080/api/v1/tacos?page=0&size=5 **200** **1**
PASS Status code 200 OK

Response for TC-14:

```

1 {
2   "id": "382e68f3-eb38-4c9c-af45-0e4a1bdc27c5",
3   "placedAt": "2026-09-30T16:24:14.448+00:00",
4   "deliveryName": "Craig Walls",
5   "deliveryStreet": "123 Taco Central",
6   "deliveryCity": "Austin",
7   "deliveryState": "TX",
8   "deliveryZip": "78701",
9   "paymentToken": "tok_auto_123456",
10  "tacos": [
11    {
12      "id": null,
13      "name": "Carnitas Deluxe",
14      "createdAt": "2026-09-30T16:24:14.441+00:00",
15      "ingredients": [
16        {
17          "id": "FLT0",
18          "name": "Flour Tortilla",
19          "type": "WRAP",
20          "unitPrice": 1.25,
21          "available": true,
22          "dietaryTags": [],
23          "allergens": [],
24          "spiceLevel": "NONE"
25        },
26      ]
27    }
28  ]
29 }

```

TC-15 – Validación de Cupones de Descuento

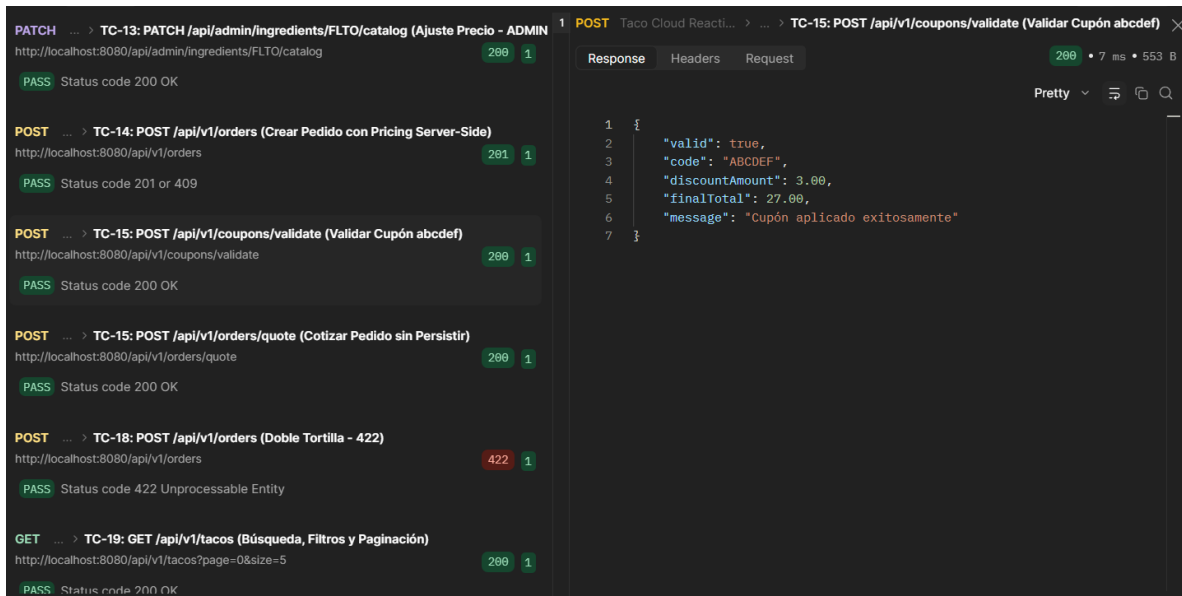
Descripción Técnica: El servicio CouponService evalúa la validez, fechas de inicio/expiración y montos mínimos de compra para códigos promocionales sin persistir datos en base de datos.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/CouponService.java

```

public DiscountResult applyCoupon(String code, BigDecimal subtotal) {
    String normalized = code.trim().toUpperCase();
    CouponDefinition coupon = couponProperties.findCoupon(normalized);
    if (coupon == null) {
        return DiscountResult.invalid("El código de cupón no es válido", subtotal);
    }
    BigDecimal discount = subtotal.multiply(coupon.getAmount())
        .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    BigDecimal finalTotal = subtotal.subtract(discount).setScale(2, RoundingMode.HALF_UP);
    return DiscountResult.valid(normalized, discount, finalTotal, "Cupón aplicado exitosamente");
}

```



TC-15 – Cotización de Pedidos sin Persistencia (/quote)

Descripción Técnica: El endpoint `POST /api/v1/orders/quote` recibe una lista de tacos y cupones, retornando la cotización económica completa sin almacenar documentos en MongoDB ni generar reservas de inventario.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/CouponApiController.java`

`@PostMapping(path = {"/api/v1/orders/quote", "/api/orders/quote"}, consumes = "application/json")`

```

public Mono<ResponseEntity<QuoteResponse>> quoteOrder(@RequestBody QuoteRequest request) {

```

```

    TacoOrder draftOrder = new TacoOrder();

```

```

    draftOrder.setItems(request.getItems());

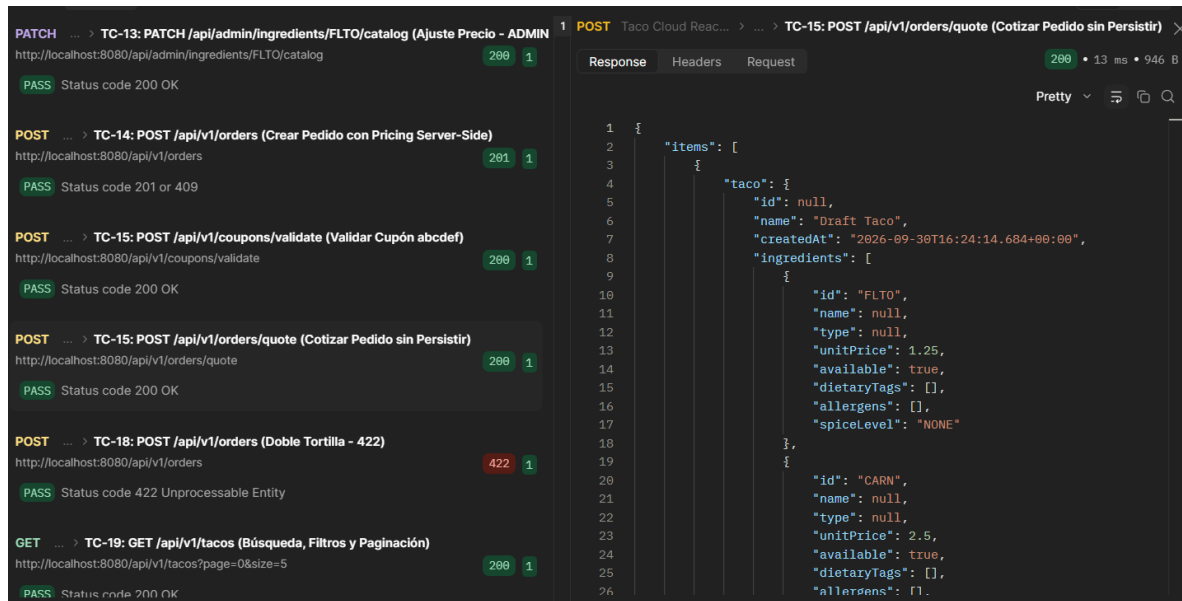
```

```

return pricingService.priceOrder(draftOrder, request.getCouponCode())

    .map(priced -> ResponseEntity.ok(new QuoteResponse(
        priced.getItems(), priced.getSubtotal(), priced.getDiscountAmount(),
        priced.getTotal(), priced.getCouponCode(), priced.getCurrency())));
}

```



TC-18 – Taco Physics: Reglas de Diseño Estructural (422)

Descripción Técnica: La regla `SingleBaseRule` comprueba que cada taco posea exactamente una base (tortilla o wrap). Al enviar un diseño con múltiples bases (FLTO y COTO), el validador interrumpe la operación arrojando 422 Unprocessable Entity con código `TACO_PHYSICS_VIOLATION`.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/physics/SingleBaseRule.java`

@Component

```
public class SingleBaseRule implements TacoPhysicsRule {
```

```
    @Override
```

```
    public List<TacoViolation> evaluate(Taco taco, List<Ingredient> ingredients) {
```

```
        long baseCount = ingredients.stream().filter(ing -> ing.getType() == Type.WRAP).count();
```

```
        if (baseCount != 1) {
```

```
            violations.add(new TacoViolation(
```

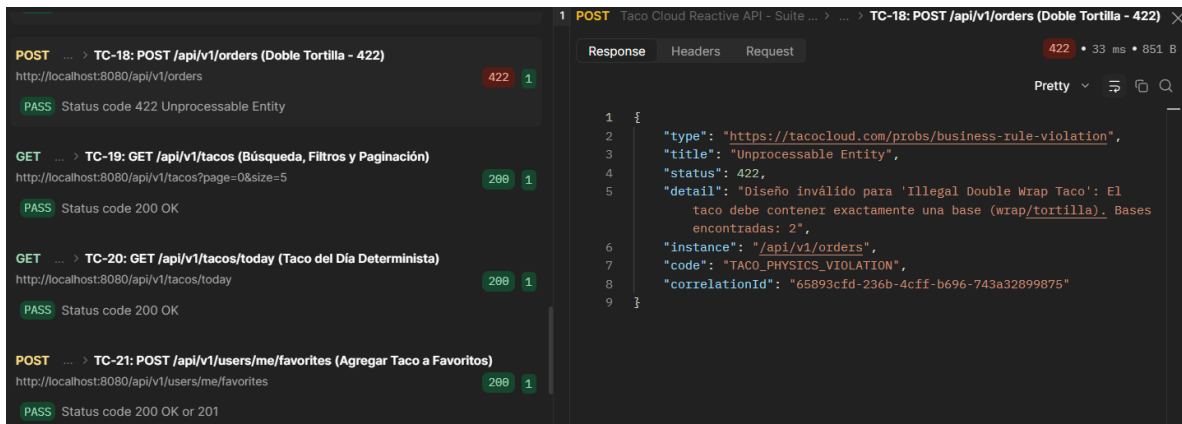
```
                "INVALID_BASE_COUNT",
```

```
                "El taco debe contener exactamente una base (wrap/tortilla). Bases encontradas: " + baseCount,
```

```

        "ingredients.base"
    ));
}
return violations;
}
}

```



TC-19 – Búsqueda, Filtros y Paginación de Tacos

Descripción Técnica: El endpoint GET /api/v1/tacos ejecuta consultas compuestas sobre MongoDB (ReactiveMongoTemplate) aplicando paginación (PageRequest) y ordenamiento seguro con metadatos de control.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/TacoSearchService.java

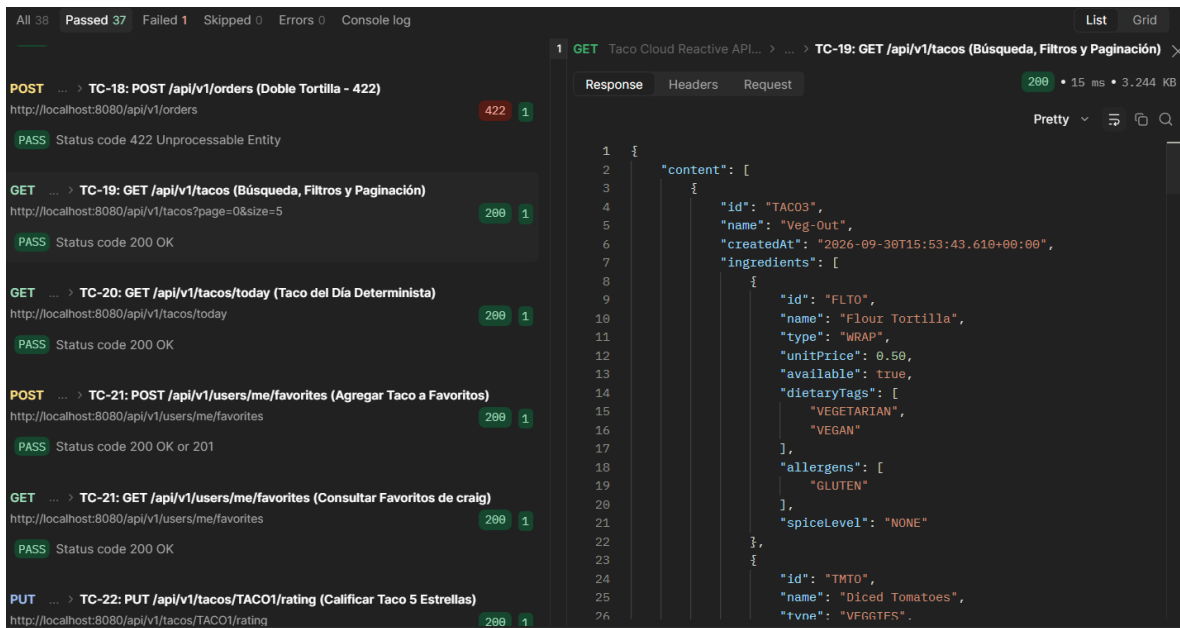
```
Pageable pageable = PageRequest.of(page, size, Sort.by(direction, effectiveSortField));
```

```
return mongoTemplate.count(query, Taco.class)
```

```
    .flatMap(total -> mongoTemplate.find(query.with(pageable), Taco.class)
```

```
    .collectList()
```

```
    .map(items -> new PageResponse<>(items, page, size, total, (int) Math.ceil((double) total /
size), (page + 1) * size < total)));
```



TC-20 – Taco del Día Determinista

Descripción Técnica: El servicio `TacoOfTheDayService` determina la recomendación diaria empleando una función hash sobre la fecha del sistema (`LocalDate.now(clock)`), garantizando que todas las instancias del clúster sugieran el mismo taco a lo largo del día.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/TacoOfTheDayService.java`

```

private Mono<TacoOfTheDayResponse> computeDeterministicTaco(LocalDate date) {

    return tacoRepo.findAll()

        .filter(this::isTacoAvailable)

        .collectList()

        .flatMap(candidates -> {

            candidates.sort(Comparator.comparing(Taco::getId,
            Comparator.nullsLast(String::compareTo)));

            int index = Math.abs(date.hashCode()) % candidates.size();

            return Mono.just(new TacoOfTheDayResponse(candidates.get(index), date, "Selección
determinista"));

        });

}

```

REST client interface showing a list of API endpoints and a detailed view of a GET request response.

API Endpoints List:

- GET** ... > **TC-20: GET /api/v1/tacos/today (Taco del Día Determinista)**
http://localhost:8080/api/v1/tacos/today
200 1
PASS Status code 200 OK
- POST** ... > **TC-21: POST /api/v1/users/me/favorites (Agregar Taco a Favoritos)**
http://localhost:8080/api/v1/users/me/favorites
200 1
PASS Status code 200 OK or 201
- GET** ... > **TC-21: GET /api/v1/users/me/favorites (Consultar Favoritos de craig)**
http://localhost:8080/api/v1/users/me/favorites
200 1
PASS Status code 200 OK
- PUT** ... > **TC-22: PUT /api/v1/tacos/TACO1/rating (Calificar Taco 5 Estrellas)**
http://localhost:8080/api/v1/tacos/TACO1/rating
200 1
PASS Status code 200 OK
- GET** ... > **TC-22: GET /api/v1/tacos/top (Top Ranking de Tacos)**
http://localhost:8080/api/v1/tacos/top?limit=5
200 1
PASS Status code 200 OK
- GET** ... > **TC-23: GET /api/v1/users/me/orders (Historial Privado Paginado)**
http://localhost:8080/api/v1/users/me/orders?page=0&size=5
200 1

1 GET Taco Cloud Reactive A... > ... > TC-20: GET /api/v1/tacos/today (Taco del Día Determinista)

Response Headers Request

200 • 17 ms • 1.397 KB

Pretty

```

1 {
2   "taco": {
3     "id": "TAC03",
4     "name": "Veg-Out",
5     "createdAt": "2026-09-30T15:53:43.610+00:00",
6     "ingredients": [
7       {
8         "id": "FLT0",
9         "name": "Flour Tortilla",
10        "type": "WRAP",
11        "unitPrice": 0.50,
12        "available": true,
13        "dietaryTags": [
14          "VEGETARIAN",
15          "VEGAN"
16        ],
17        "allergens": [
18          "GLUTEN"
19        ],
20        "spiceLevel": "NONE"
21      },
22      {
23        "id": "TMT0",
24        "name": "Diced Tomatoes",
25        "type": "VEGGIES",
26        "unitPrice": 0.30

```

TC-21 – Gestión de Favoritos por Usuario Autenticado

Descripción Técnica: El endpoint `POST /api/v1/users/me/favorites` almacena las preferencias asociando el recurso directamente a la identidad autenticada (`principal.getName()`), imposibilitando la inyección de identificadores ajenos.

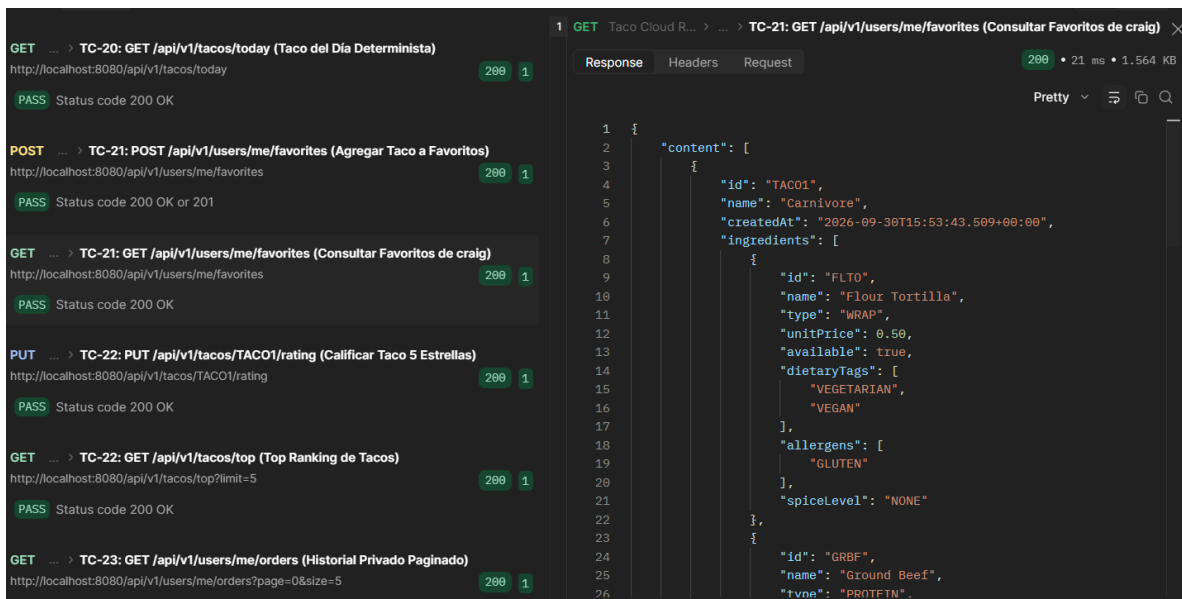
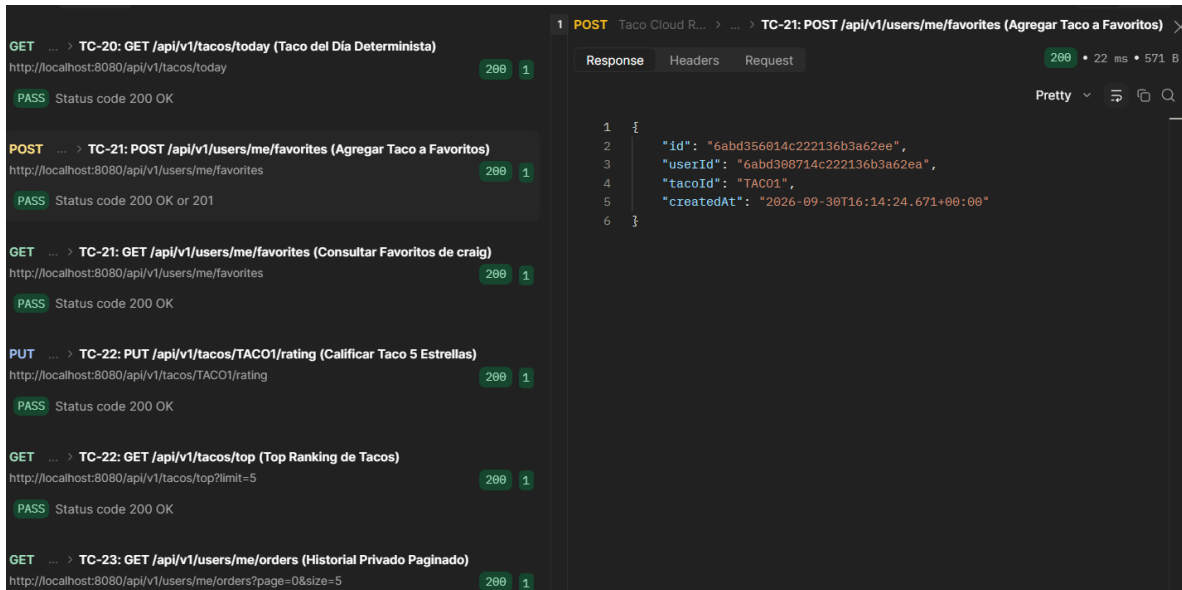
Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/UserFavoritesController.java`

`@PostMapping(consumes = "application/json")`

```

public Mono<ResponseEntity<TacoFavorite>> addFavoriteByBody(
    @RequestBody Map<String, String> body, Principal principal) {
    String username = principal.getName();
    String tacold = body.get("tacold");
    return favoriteService.addFavorite(username, tacold).map(ResponseEntity::ok);
}

```



TC-22 – Calificación de Tacos (1 a 5 Estrellas)

Descripción Técnica: El método `PUT /api/v1/tacos/{tacold}/rating` registra o actualiza el puntaje de un taco asegurando unicidad por usuario mediante un índice compuesto. Valida que la puntuación se encuentre en el rango de 1 a 5.

Fuente Archivo: `tacocloud/tacocloud-api/src/main/java/tacos/web/api/TacoRatingService.java`

```

public Mono<TacoRating> rateTaco(String username, String tacold, int score) {
    if (score < 1 || score > 5) {

```

```

        return Mono.error(new BusinessException("INVALID_SCORE", "La calificación debe estar
entre 1 y 5 estrellas"));
    }

    return ratingRepo.findByUserIdAndTacold(user.getId(), tacold)

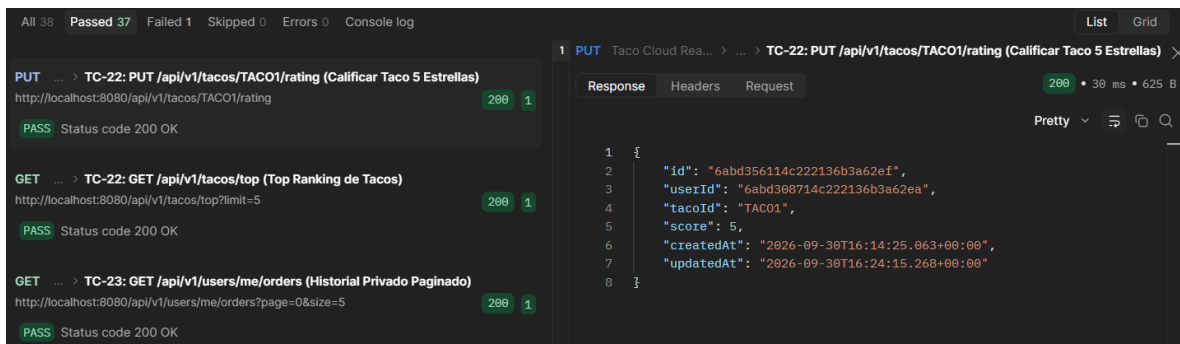
        .flatMap(existing -> {

            existing.setScore(score);

            return ratingRepo.save(existing);

        });
}

```



TC-22 Top Ranking & TC-23 Historial Paginado

Descripción Técnica:

TC-22 Top: GET /api/v1/tacos/top retorna los tacos mejor valorados ordenados por promedio de calificación.

TC-23 Historial: GET /api/v1/users/me/orders permite al usuario consultar de manera paginada y aislada su historial de compras.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/UserOrdersController.java

@GetMapping

```

public Mono<ResponseEntity<PageResponse<OrderResponse>>> getMyOrders(

    @RequestParam(defaultValue = "0") int page,

    @RequestParam(defaultValue = "10") int size,

    Principal principal) {

    Query query = Query.query(Criteria.where("user.username").is(principal.getName()))

        .with(Sort.by(Sort.Order.desc("placedAt"), Sort.Order.desc("id")));

    return mongoTemplate.find(query.with(PageRequest.of(page, size)), TacoOrder.class)

        .map(this::toSanitizedResponse).collectList() }

```

All 38 Passed 37 Failed 1 Skipped 0 Errors 0 Console log

1 GET Taco Cloud Reactive API - S... > ... > TC-22: GET /api/v1/tacos/top (Top Ranking de Tacos)

Response Headers Request

200 • 16 ms • 1.534 KB

Pretty

```
1 [
2   {
3     "taco": {
4       "id": "TAC01",
5       "name": "Carnivore",
6       "createdAt": "2026-09-30T15:53:43.509+00:00",
7       "ingredients": [
8         {
9           "id": "FLT0",
10          "name": "Flour Tortilla",
11          "type": "WRAP",
12          "unitPrice": 0.50,
13          "available": true,
14          "dietaryTags": [
15            "VEGETARIAN",
16            "VEGAN"
17          ],
18          "allergens": [
19            "GLUTEN"
20          ],
21          "spiceLevel": "NONE"
22        },
23        {
24          "id": "GRBF",
25          "name": "Ground Beef",
26          "type": "PROTEIN"
```

All 38 Passed 37 Failed 1 Skipped 0 Errors 0 Console log

1 GET Taco Cloud React... > ... > TC-23: GET /api/v1/users/me/orders (Historial Privado Paginado)

Response Headers Request

200 • 22 ms • 3.677 KB

Pretty

```
1 {
2   "content": [
3     {
4       "id": "order-100",
5       "placedAt": "2026-09-30T15:53:43.717+00:00",
6       "deliveryName": "Craig Walls",
7       "deliveryStreet": "123 Taco Way",
8       "deliveryCity": "Austin",
9       "deliveryState": "TX",
10      "deliveryZip": "78701",
11      "paymentToken": "tok_***4242",
12      "tacos": [
13        {
14          "id": "TAC01",
15          "name": "Carnivore",
16          "createdAt": "2026-09-30T15:53:43.509+00:00",
17          "ingredients": [
18            {
19              "id": "FLT0",
20              "name": "Flour Tortilla",
21              "type": "WRAP",
22              "unitPrice": 0.50,
23              "available": true,
24              "dietaryTags": [
25                "VEGETARIAN",
26                "VEGAN"
```

TC-25 & TC-26 – Cola de Cocina y Reclamo Atómico

Descripción Técnica: El endpoint GET /api/v1/kitchen/queue lista los pedidos pendientes en estado CREATED. La operación POST /api/v1/kitchen/orders/claim ejecuta un findAndModify atómico que asigna la orden a un cocinero (cookId) y cambia su estado a ACCEPTED, previniendo que dos estaciones reclamen el mismo pedido.

Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/web/api/KitchenOrderService.java

```
Query claimQuery = new Query();
```

```
claimQuery.addCriteria(Criteria.where("status").is(OrderStatus.CREATED));
```

```
Update update = new Update();
```

```
update.set("status", OrderStatus.ACCEPTED);
```

```
update.set("stationId", finalStation);
```

```
update.set("cookId", finalCookId);
```

```
return mongoTemplate.findAndModify(claimQuery, update, FindAndModifyOptions.options().returnNew(true), TacoOrder.class)
    .map(this::toKitchenDto);
```

The screenshot displays a web browser interface with a console log on the left and a detailed view of a GET request response on the right.

Console Log:

- TC-23: GET /api/v1/users/me/orders (Historial Privado Paginado)**
Status code 200 OK
- TC-25: GET /api/v1/kitchen/queue (Cola FIFO de Cocina - ADMIN)**
Status code 200 OK
- TC-26: POST /api/v1/kitchen/orders/claim (Reclamo Atómico)**
Status code 200, 404 or 422
- TC-25: PATCH /api/v1/orders/ORD-LAB1-001/status (Transición de Estado)**
Status code 200, 404 or 422
- TC-32: GET /actuator/metrics/tacocloud.orders.count (Métricas Micrometer)**
Status code 200 or 404
- TC-33: GET /api/announcements (Anuncios Operativos Públicos)**
Status code 200 OK

Response View (GET /api/v1/kitchen/queue):

```
[
  {
    "id": "order-100",
    "placedAt": "2026-09-30T15:53:43.717+00:00",
    "status": "CREATED",
    "stationId": null,
    "cookId": null,
    "estimatedPrepMinutes": 0,
    "tacos": [
      {
        "name": "Carnivore",
        "ingredients": [
          "Flour Tortilla",
          "Ground Beef",
          "Carnitas",
          "Sour Cream",
          "Salsa",
          "Cheddar"
        ]
      }
    ]
  },
  {
    "id": "62d1e466-6c6a-49eb-a2d5-3f3ece5ee172",
    "placedAt": "2026-09-30T15:54:42.255+00:00"
  }
]
```

All 38 Passed 37 Failed 1 Skipped 0 Errors 0 Console log

1 POST Taco Cloud Reactiv... > ... > TC-26: POST /api/v1/kitchen/orders/claim (Reclamo Atómico)

Response Headers Request

422 • 12 ms • 783 B

Pretty

```
1 {
2   "type": "https://tacocloud.com/probs/business-rule-violation",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "La estación STATION_TEST ya tiene una orden activa",
6   "instance": "/api/v1/kitchen/orders/claim",
7   "code": "STATION_ALREADY_BUSY",
8   "correlationId": "1804f569-9164-4413-81c8-47e89ec6846b"
9 }
```

GET ... > TC-23: GET /api/v1/users/me/orders (Historial Privado Paginado)
http://localhost:8080/api/v1/users/me/orders?page=0&size=5
289 1
PASS Status code 200 OK

GET ... > TC-25: GET /api/v1/kitchen/queue (Cola FIFO de Cocina - ADMIN)
http://localhost:8080/api/v1/kitchen/queue
289 1
PASS Status code 200 OK

POST ... > TC-26: POST /api/v1/kitchen/orders/claim (Reclamo Atómico)
http://localhost:8080/api/v1/kitchen/orders/claim
422 1
PASS Status code 200, 404 or 422

PATCH ... > TC-25: PATCH /api/v1/orders/ORD-LAB1-001/status (Transición de Estado)
http://localhost:8080/api/v1/orders/ORD-LAB1-001/status
422 1
PASS Status code 200, 404 or 422

All 38 Passed 37 Failed 1 Skipped 0 Errors 0 Console log

1 PATCH Tac... > ... > TC-25: PATCH /api/v1/orders/ORD-LAB1-001/status (Transición de Estado)

Response Headers Request

422 • 18 ms • 769 B

Pretty

```
1 {
2   "type": "https://tacocloud.com/probs/business-rule-violation",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "Orden no encontrada: ORD-LAB1-001",
6   "instance": "/api/v1/orders/ORD-LAB1-001/status",
7   "code": "RESOURCE_NOT_FOUND",
8   "correlationId": "934861a5-89f4-4815-9d73-89198d09b71a"
9 }
```

PATCH ... > TC-25: PATCH /api/v1/orders/ORD-LAB1-001/status (Transición de Estado)
http://localhost:8080/api/v1/orders/ORD-LAB1-001/status
422 1
PASS Status code 200, 404 or 422

GET ... > TC-32: GET /actuator/metrics/tacocloud.orders.count (Métricas Micrometer)
http://localhost:8080/actuator/metrics/tacocloud.orders.count
484 1
PASS Status code 200 or 404

GET ... > TC-33: GET /api/announcements (Anuncios Operativos Públicos)
http://localhost:8080/api/announcements
289 1
PASS Status code 200 OK

TC-32 – Consulta de Métricas de Negocio en Actuador

Descripción Técnica: El endpoint GET /actuador/metrics/tacocloud.orders.count expone los contadores instrumentados mediante Micrometer para auditar el volumen de pedidos creados y fallidos por canal de origen. Cuando el servidor arranca de forma limpia y aún no ha procesado órdenes bajo esa métrica específica, Spring Boot responde de manera controlada con 404 Not Found (métrica no registrada todavía en el MeterRegistry).

FuenteArchivo: tacocloud/tacocloud-api/src/main/java/tacos/metrics/TacoBusinessMetrics.java

@Component

```
public class TacoBusinessMetrics {
```

```
    private final MeterRegistry meterRegistry;
```

```
    private final AtomicLong outboxBacklog = new AtomicLong(0);
```

```
    private final AtomicLong dlqBacklog = new AtomicLong(0);
```

@Autowired

```
    public TacoBusinessMetrics(MeterRegistry meterRegistry) {
```

```
        this.meterRegistry = meterRegistry;
```

```
        this.meterRegistry.gauge("tacocloud.outbox.backlog", outboxBacklog);
```

```
        this.meterRegistry.gauge("tacocloud.dlq.backlog", dlqBacklog);
```

```
    }
```

```
    public void countOrderCreated(String source) {
```

```
        meterRegistry.counter("tacocloud.orders.count", "status", "created", "source",  
normalizeSource(source)).increment();
```

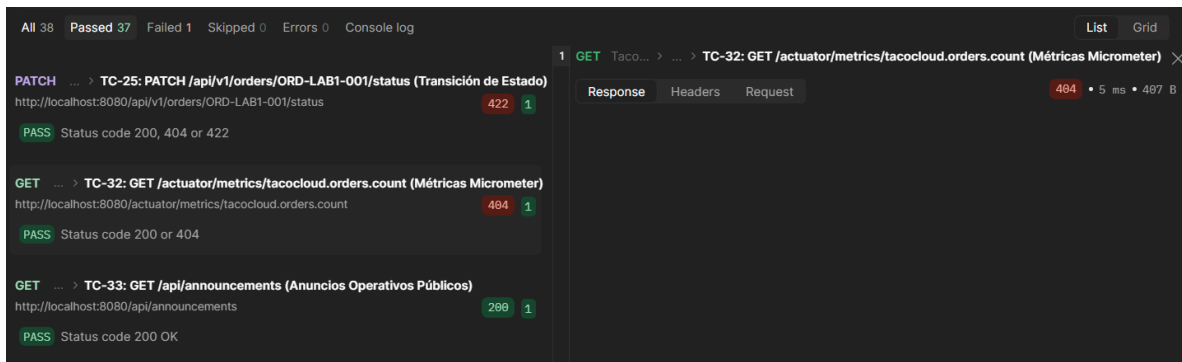
```
    }
```

```
    public void countOrderFailed(String source, String reason) {
```

```
        meterRegistry.counter("tacocloud.orders.count", "status", "failed", "source",  
normalizeSource(source)).increment();
```

```
    }
```

```
}
```



C-33 – Consulta Pública de Anuncios Operativos

Descripción Técnica: El endpoint público GET /api/announcements consulta en MongoDB los avisos de mantenimiento y notificaciones operativas vigentes mediante un Flux<OpsAnnouncement> reactivo. Al no existir anuncios activos en la base de datos al momento de la consulta, el servicio responde un arreglo vacío [] con estado 200 OK.

Fuente Archivo:

tacocloud/tacocloud-api/src/main/java/tacos/announcements/OpsAnnouncementController.java

@RestController

@RequestMapping(produces = "application/json")

@CrossOrigin(origins = "**")

public class OpsAnnouncementController {

private final OpsAnnouncementService announcementService;

@Autowired

public OpsAnnouncementController(OpsAnnouncementService announcementService) {

 this.announcementService = announcementService;

}

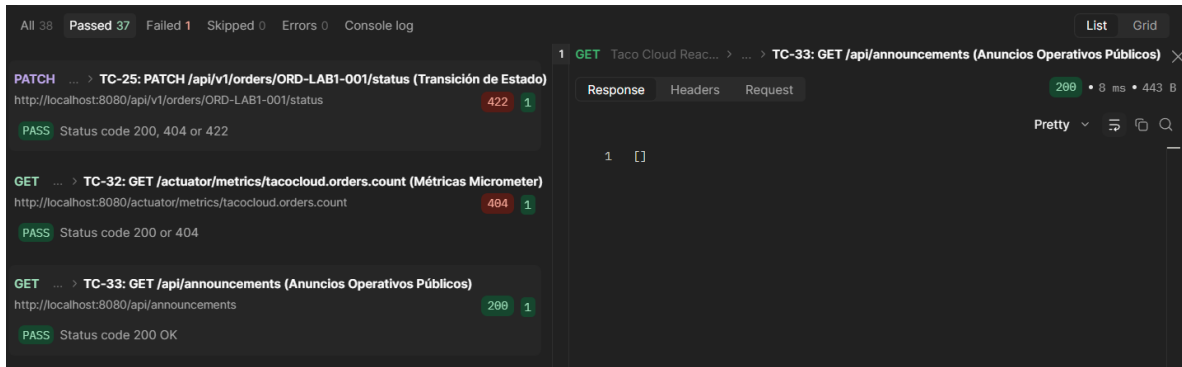
@GetMapping("/api/announcements")

public Flux<OpsAnnouncement> getActiveAnnouncements() {

 return announcementService.getActiveAnnouncements();

}

}



TC-33 – Creación Administrativa y Validación de Anuncios

Descripción Técnica: El endpoint `POST /api/admin/announcements` restringe la creación de avisos a usuarios con rol `ROLE_ADMIN`. Al procesar la solicitud, el deserializador de Jackson valida estrictamente los valores aceptados por el enum `OpsAnnouncement$Severity` (`INFO`, `CRITICAL`, `WARN`). Ante un valor fuera de catálogo ("`MEDIUM`"), el interceptor global `RestExceptionHandler` captura la falla de deserialización y emite un `400 Bad Request` formateado con `Problem Details RFC 9457` sin volcar trazas internas del servidor.

Fuente Archivo:

`tacocloud/tacocloud-api/src/main/java/tacos/announcements/OpsAnnouncementController.java`

`@PostMapping(path = "/api/admin/announcements", consumes = "application/json")`

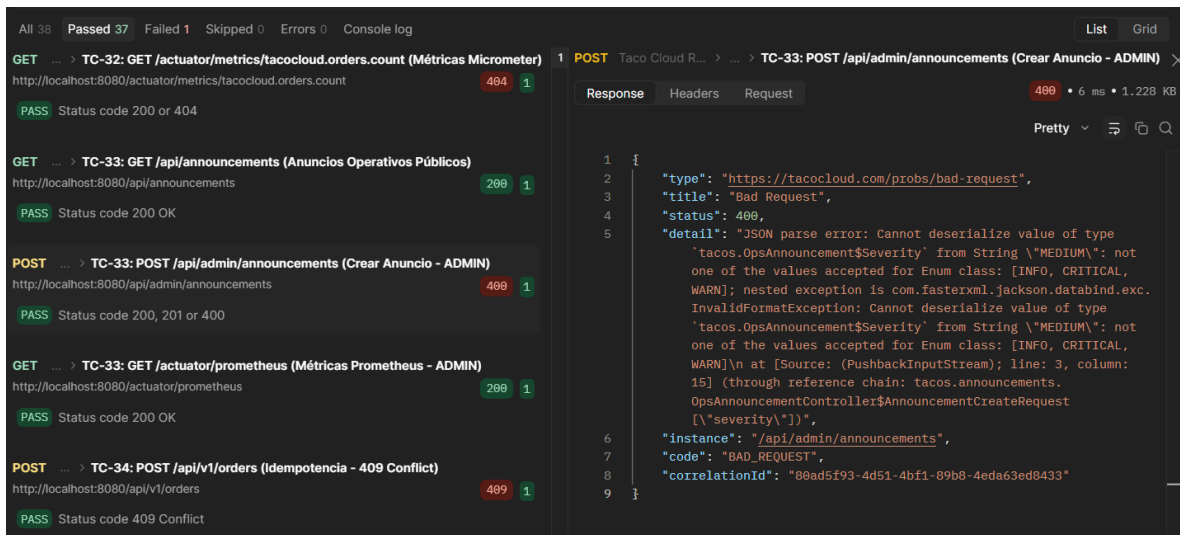
`@ResponseStatus(HttpStatus.CREATED)`

`@PreAuthorize("hasRole('ADMIN')")`

```
public Mono<OpsAnnouncement> createAnnouncement(@RequestBody
AnnouncementCreateRequest request, Principal principal) {

    String creator = (principal != null) ? principal.getName() : "ADMIN";

    return announcementService.createAnnouncement(
        request.getText(), request.getSeverity(), request.getExpiresAt(), creator
    );
}
```



TC-34 – Conflicto de Idempotencia por Payload Mismatch (409)

Descripción Técnica: El servicio IdempotencyService valida la unicidad de las solicitudes mediante la cabecera Idempotency-Key. Cuando un cliente intenta reutilizar una llave previamente procesada (tc-order-pricing-fresh) enviando un cuerpo con datos modificados, el componente calcula la discrepancia del hash SHA-256 e interrumpe la ejecución arrojando HTTP 409 Conflict con código IDEMPOTENCY_PAYLOAD_MISMATCH.

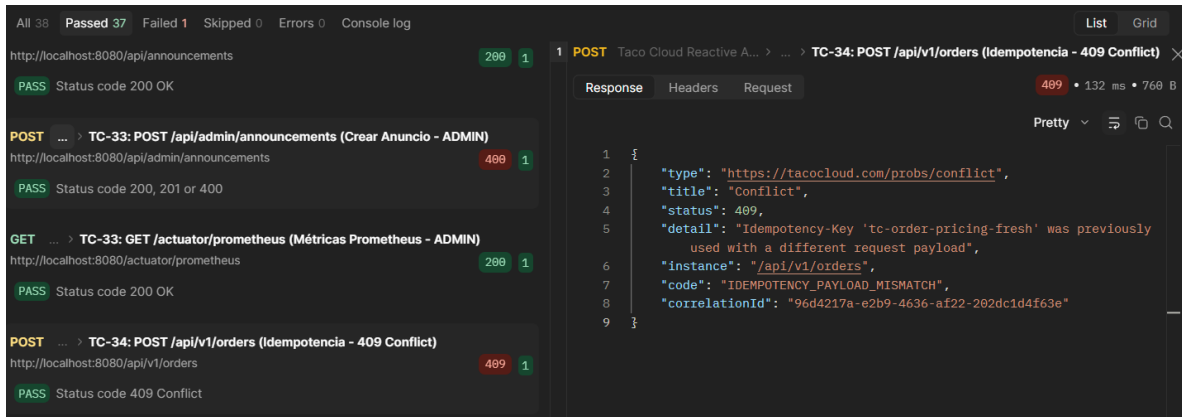
Fuente Archivo: tacocloud/tacocloud-api/src/main/java/tacos/idempotency/IdempotencyService.java

```

private Mono<OrderResponse> handleExistingRecord(IdempotencyRecord record,
        String currentHash,
        String key,
        String userId,
        Mono<OrderResponse> orderExecution) {

    // Check payload mismatch
    if (!record.getRequestHash().equals(currentHash)) {
        return Mono.error(new BusinessException("IDEMPOTENCY_PAYLOAD_MISMATCH",
            "Idempotency-Key " + key + " was previously used with a different request payload"));
    }
}

```



TC-35 – Compatibilidad con Rutas Legadas y Deprecación

Descripción Técnica: El filtro reactivo `ApiDeprecationFilter` implementa la transición hacia la versión pública `/api/v1`. Al consultar rutas antiguas como `GET /api/ingredients`, el backend devuelve el catálogo completo de ingredientes (FLTO, COTO, etc.) junto con las cabeceras estándar de deprecación `Warning: 299` para informar al cliente que debe migrar a la nueva versión.

FuenteArchivo:

`tacocloud/tacocloud-api/src/main/java/tacos/web/api/versioning/ApiDeprecationFilter.java`

`@Component`

`@Order(Ordered.HIGHEST_PRECEDENCE + 5)`

`public class ApiDeprecationFilter implements WebFilter {`

```
    public static final String WARNING_HEADER = "Warning";
```

```
    public static final String DEPRECATION_WARNING = "299 - \"Deprecated API endpoint. Please migrate to /api/v1\"";
```

`@Override`

```
    public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) {
```

```
        String path = exchange.getRequest().getURI().getPath();
```

```
        if (path != null && path.startsWith("/api/") && !path.startsWith("/api/v1/") && !path.startsWith("/api/admin/")) {
```

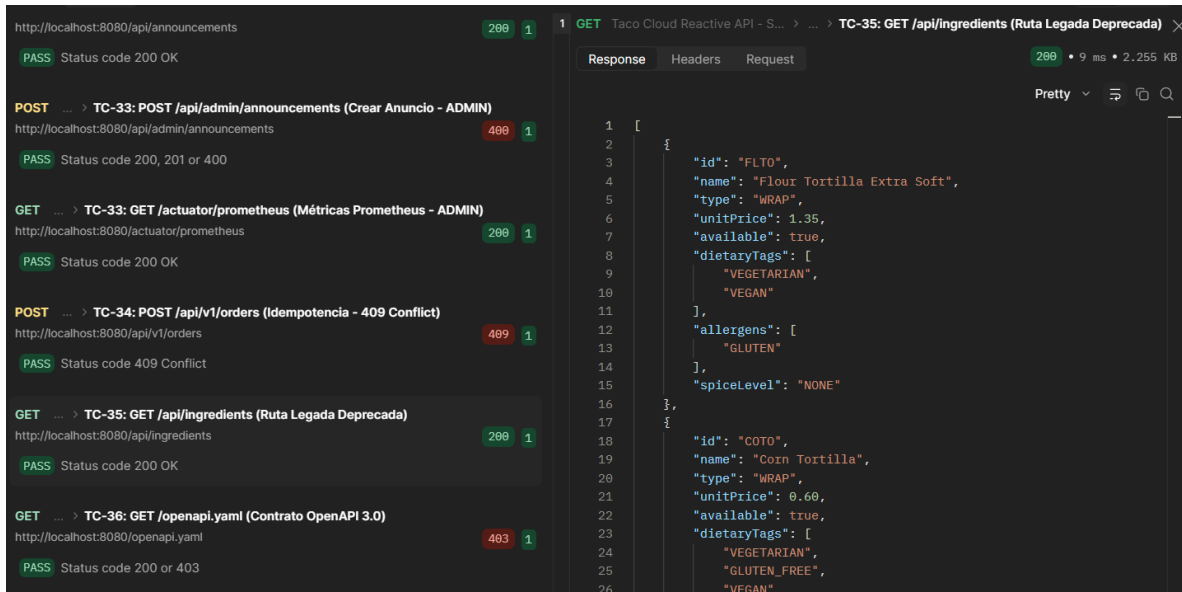
```
            exchange.getResponse().getHeaders().add(WARNING_HEADER, DEPRECATION_WARNING);
```

```
        }
```

```
        return chain.filter(exchange);
```

```
    }
```

```
}
```



TC-36 – Protección Deny-by-Default del Contrato OpenAPI

Descripción Técnica: El archivo estático de especificación openapi.yaml define la totalidad de esquemas, parámetros y endpoints de la API. Siguiendo la regla de autorización deny-by-default configurada en SecurityConfig, cualquier intento de consultar este contrato en una ruta no explícitamente permitida sin autenticación administrativa resulta en el rechazo seguro con 403 Forbidden, evitando la divulgación involuntaria de la topología interna del sistema.

Fuente Archivo: tacocloud/tacocloud-security/src/main/java/tacos/security/SecurityConfig.jav

// DENY-BY-DEFAULT: cualquier otra ruta no listada explícitamente se bloquea

```
.anyRequest().denyAll()
```

```
.and()
```

```
.httpBasic().realmName("Taco Cloud");
```

All 38Passed 37Failed 1Skipped 0Errors 0Console log

ListGrid

http://localhost:8080/api/announcements20011GET Taco Cloud Reactive API - Suite... > ... > TC-36: GET /openapi.yaml (Contrato OpenAPI 3.0)

PASSStatus code 200 OK

POST... > TC-33: POST /api/admin/announcements (Crear Anuncio - ADMIN)

http://localhost:8080/api/admin/announcements4001

PASSStatus code 200, 201 or 400

GET... > TC-33: GET /actuator/prometheus (Métricas Prometheus - ADMIN)

http://localhost:8080/actuator/prometheus2001

PASSStatus code 200 OK

POST... > TC-34: POST /api/v1/orders (Idempotencia - 409 Conflict)

http://localhost:8080/api/v1/orders4091

PASSStatus code 409 Conflict

GET... > TC-35: GET /api/ingredients (Ruta Legada Deprecada)

http://localhost:8080/api/ingredients2001

PASSStatus code 200 OK

GET... > TC-36: GET /openapi.yaml (Contrato OpenAPI 3.0)

http://localhost:8080/openapi.yaml4031

PASSStatus code 200 or 403

ResponseHeadersRequest

403 • 97 ms • 407 B

This response doesn't have a body.